

CO1

ASSESSMENT TOOL-1

NAME : P. Ranjithkumar

DATE:13.07.2026

REG.NO : 192411119

MONDAY

COURSE CODE : CSA1103

COURSE NAME : OOAD

1: Construct a Concept Map

Given

Requirement Analysis → _____ → Coding → Testing → _____ → Maintenance

Completed Concept Map

Requirement Analysis



Design



Coding



Testing



Deployment



Maintenance

Missing Concepts

- Design
- Deployment

Explanation

The **Software Development Life Cycle (SDLC)** is a structured process followed by software engineers to develop high-quality software. Every phase in SDLC has a specific purpose and contributes to the successful completion of the project.

Design Phase

The Design phase starts after the Requirement Analysis phase. In this phase, developers and system architects convert the user requirements into a detailed technical design or blueprint. This blueprint explains how the software will be developed, including the system architecture, database design, user interface, module structure, algorithms, and technology stack.

The Design phase is generally divided into:

- **High-Level Design (HLD):** Defines the overall architecture of the software.
- **Low-Level Design (LLD):** Describes the internal details of each module.

A well-designed system reduces errors during coding and makes the software easier to understand and maintain.

Deployment Phase

Deployment is the process of releasing the completed software to the users after successful testing. During deployment, the software is installed in the production environment and made available to end users.

Deployment activities include:

- Installing the application
- Configuring servers
- Migrating databases
- Providing user training (if necessary)
- Monitoring software performance after release

A successful deployment ensures that the software works correctly in the real-world environment and meets customer expectations.

Analysis / Importance

- SDLC provides a systematic approach to software development.
- Design creates a clear roadmap before coding begins.
- Good design minimizes coding errors and improves software quality.
- Deployment ensures the software reaches the intended users successfully.

- Proper planning reduces project risks and development costs.
- Following SDLC improves project management and team coordination.
- It enhances software reliability, maintainability, and scalability.
- Every phase contributes to the successful completion of the project.
- Maintenance becomes easier when the software is properly designed.
- Overall, SDLC helps in delivering high-quality software within time and budget.

Question 2: Construct a Concept Map

Given:

Object-Oriented Principles

→ Abstraction

→ _____

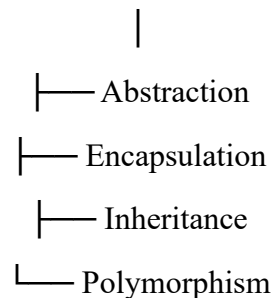
→ Inheritance

→ _____

Answer

Completed Concept Map

Object-Oriented Principles



Missing Concepts

- Encapsulation
- Polymorphism

Explanation

Object-Oriented Programming (OOP) is a programming methodology that organizes software into objects. It is based on four important principles that improve software quality, security, and reusability.

Encapsulation

Encapsulation is the process of combining data (variables) and methods (functions) into a single unit called a class. It also hides internal data from unauthorized access using access modifiers such as private, protected, and public.

For example, in a Bank Account class, the account balance is kept private, and users can access it only through methods like deposit() and withdraw(). This prevents accidental modification of sensitive data.

Polymorphism

Polymorphism means "**many forms.**" It allows the same method or interface to perform different tasks depending on the object.

For example:

- A person can behave differently as a Student, Teacher, or Employee.
- In programming, one method name can perform different operations based on different objects.

Polymorphism makes programs flexible, reusable, and easy to extend.

Analysis / Importance

- Encapsulation protects sensitive data from unauthorized access.
- It improves software security and data integrity.
- Polymorphism allows one interface to support multiple implementations.
- OOP principles reduce code duplication.
- They improve software flexibility and maintainability.
- Code becomes easier to modify and debug.
- Supports real-world problem modeling.
- Enhances software scalability.
- Encourages reusable programming components.
- Makes large software projects easier to manage.

Question 3: Construct a Concept Map

Given:

Class → Blueprint

Object → Instance

Method → _____

Attribute → _____

Answer

Completed Concept Map

Class → Blueprint

Object → Instance

Method → Behavior

Attribute → State

Missing Concepts

- Behavior
- State

Explanation

Object-Oriented Programming is based on the concepts of classes and objects. Every object has characteristics and actions that help represent real-world entities.

Method → Behavior

A method is a function defined inside a class. Methods specify the actions or behaviors that an object can perform.

For example:

A Car object may contain methods like:

- Start()
- Stop()
- Accelerate()
- Brake()

These methods describe the behavior of the car.

Attribute → State

Attributes are variables that store information about an object. They represent the current condition or state of the object.

For example:

A Student object may have attributes such as:

- Name
- Roll Number
- Age
- Department
- Marks

The values stored in these attributes define the current state of the student object.

Analysis / Importance

- Classes act as blueprints for creating objects.
- Objects represent real-world entities.
- Methods define what an object can perform.
- Attributes store object information.
- Together they improve software organization.
- They support modular programming.
- Objects simplify complex systems.
- Classes improve code reusability.
- Programs become easier to maintain.
- OOP improves software development efficiency.

Question 4: A Concept Map Shows

Given:

Software Quality

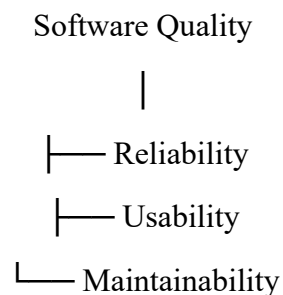
→ Reliability

→ _____

→ Maintainability

Answer

Completed Concept Map



Missing Concept

- Usability

Explanation

Software quality refers to the ability of software to satisfy user requirements while providing reliable and efficient performance. Three important quality attributes shown in this concept map are Reliability, Usability, and Maintainability.

Reliability

Reliability is the ability of software to perform its intended functions correctly without failure for a specified period under given conditions. Reliable software produces accurate results consistently and minimizes system crashes.

Usability

Usability measures how easy the software is to learn, understand, and use. A software application with a simple interface, clear navigation, and helpful instructions provides a better user experience.

Good usability reduces training time and increases user satisfaction.

Maintainability

Maintainability refers to the ease with which software can be modified, corrected, updated, or improved after deployment. Well-written and well-documented software is easier to maintain.

Maintainable software reduces maintenance costs and increases the software's lifespan.

Analysis / Importance

- Reliability ensures consistent software performance.
- Usability improves customer satisfaction and user experience.
- Maintainability simplifies software updates and bug fixing.
- High-quality software reduces operational failures.
- Quality software increases customer trust.
- It reduces maintenance and support costs.
- Improves overall software efficiency.
- Supports long-term software evolution.
- Enhances software acceptance in the market.
- Software quality is one of the most important goals of Software Engineering.

Question 5: UML Concept Map

Given:

UML

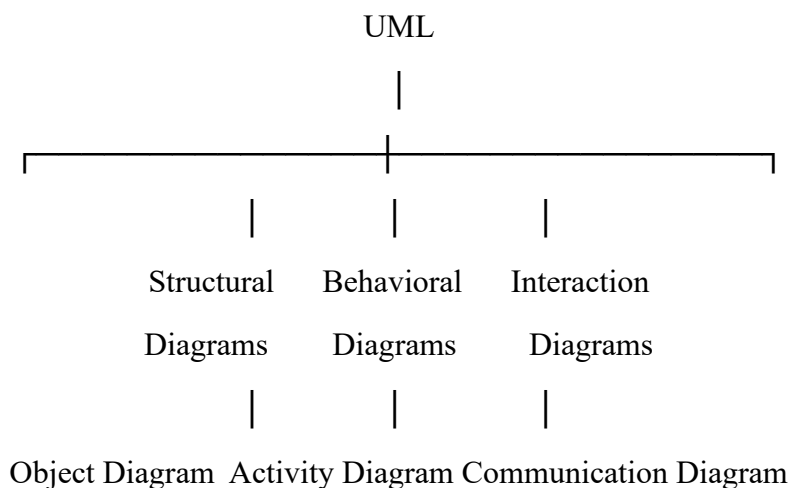
→ Structural Diagrams → _____

→ Behavioral Diagrams → Activity Diagram

→ Interaction Diagrams → _____

Answer

Completed Concept Map



Missing Concepts

- Object Diagram
- Communication Diagram

Explanation

Unified Modeling Language (UML) is a standardized visual modeling language used in Software Engineering to represent, design, and document software systems. UML provides different types of diagrams to describe various aspects of a software project.

Object Diagram

An Object Diagram is a type of structural diagram that represents the objects of a system at a particular moment in time. It shows the instances of classes and the relationships between those objects. Unlike a Class Diagram, which describes the blueprint of a system, an Object Diagram represents the actual objects created during execution.

Object diagrams are useful for:

- Visualizing object relationships.
- Understanding the runtime state of the system.
- Verifying the design of class diagrams.

Communication Diagram

A Communication Diagram is an interaction diagram that illustrates how objects communicate with each other by exchanging messages. It focuses on object collaboration rather than the sequence of events.

Communication diagrams help developers understand:

- Which objects interact.
- The messages exchanged.
- The responsibilities of each object.

Analysis / Importance

- UML provides a standard way to visualize software systems.
- Object diagrams represent real-time object instances.
- Communication diagrams show collaboration among objects.
- UML improves communication between developers and clients.
- It reduces misunderstandings during development.
- UML helps in system documentation.
- It supports object-oriented analysis and design.
- UML diagrams simplify complex software systems.
- They improve software quality and maintainability.
- UML is widely used in software industries.

Question 6: Identify the Failure Phase

Given:

Requirement Analysis ✓

Design ✓

Coding ✗

Testing ✗

Deployment ✗

Answer

Failure Phase

Coding Phase

Explanation

The Coding phase is the stage where developers convert the software design into executable source code using programming languages such as Java, Python, or C++.

In the given SDLC process, the Coding phase has failed. This means developers have introduced errors while implementing the software. These errors may include syntax errors, logical mistakes, incorrect algorithms, improper variable usage, or failure to follow coding standards.

Because coding is incomplete or incorrect, Testing cannot begin successfully. As a result, Deployment is also delayed because the software is not ready for release.

The Coding phase acts as the foundation for the remaining phases of software development. If coding contains defects, they propagate to later stages, increasing both development time and cost.

Analysis / Impact

- Coding errors reduce software quality.
- Logical mistakes produce incorrect outputs.
- Testing becomes difficult or impossible.
- Project deadlines may be delayed.
- Development costs increase.
- Customer satisfaction decreases.
- More debugging effort is required.
- Software reliability is affected.
- Maintenance becomes more difficult.

Solution

- Follow coding standards.
- Perform code reviews regularly.
- Conduct unit testing.
- Use debugging tools.
- Write clear and modular code.
- Maintain proper documentation.

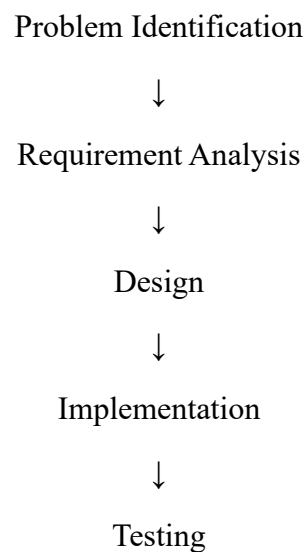
Question 7: Construct a Concept Map

Given:

Problem Identification → _____ → Design → Implementation → _____

Answer

Completed Concept Map



Missing Concepts

- Requirement Analysis
- Testing

Explanation

Software development begins by identifying the problem that needs to be solved. After understanding the problem, Requirement Analysis is performed to collect and document all user requirements.

During Requirement Analysis, software engineers communicate with clients and stakeholders to understand what the software should do. Both functional and non-functional requirements are identified and documented.

After designing and implementing the software, Testing is performed to verify that the developed system satisfies all specified requirements. Testing identifies defects, verifies functionality, and ensures the software behaves as expected before deployment.

Analysis / Importance

- Requirement Analysis identifies customer needs.
- It prevents misunderstandings during development.
- Proper requirements reduce future modifications.
- Testing identifies software defects.
- Testing improves reliability.
- Early testing reduces maintenance costs.
- Software quality improves significantly.
- Customer requirements are validated.
- Project success rate increases.
- High-quality software is delivered.

Question 8: Construct a Concept Map

Given:

Object-Oriented Features

→ Class

→ _____

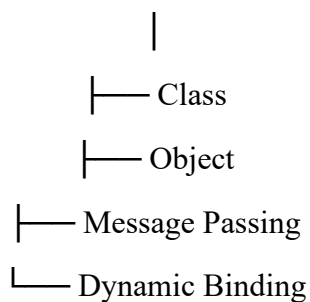
→ Message Passing

→ _____

Answer

Completed Concept Map

Object-Oriented Features



Missing Concepts

- Object
- Dynamic Binding

Explanation

Object-Oriented Programming (OOP) provides several features that make software modular, reusable, and flexible.

Object

An Object is an instance of a class. It contains both data (attributes) and methods (functions). Objects represent real-world entities such as students, employees, bank accounts, or vehicles.

For example, a Student class can create multiple Student objects with different names and roll numbers.

Dynamic Binding

Dynamic Binding is the process in which the method to be executed is determined during runtime instead of compile time. It allows the program to decide which method should be called based on the actual object.

Dynamic Binding supports runtime polymorphism and increases software flexibility.

Analysis / Importance

- Objects represent real-world entities.
- Classes act as templates for objects.
- Dynamic Binding enables runtime method selection.
- It improves software flexibility.
- Supports polymorphism.
- Reduces code duplication.
- Makes software scalable.
- Simplifies maintenance.
- Encourages reusable programming.
- Improves software performance.

Question 9: Construct a Concept Map

Given:

Software Development Models

→ Waterfall

→ _____

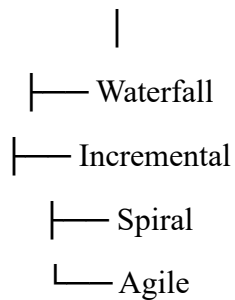
→ Spiral

→ _____

Answer

Completed Concept Map

Software Development Models



Missing Concepts

- Incremental Model
- Agile Model

Explanation

Software Development Models provide different approaches for planning, developing, testing, and maintaining software systems.

Incremental Model

The Incremental Model develops software in multiple small increments. Each increment adds new functionality to the existing system until the complete software is developed.

Advantages include:

- Faster delivery
- Easier testing
- Lower development risk

Agile Model

The Agile Model is an iterative software development methodology that focuses on continuous customer involvement, teamwork, and frequent software releases.

Agile allows developers to respond quickly to changing customer requirements and encourages regular feedback throughout development.

Analysis / Importance

- Waterfall is suitable for fixed requirements.
- Incremental Model delivers software in stages.
- Spiral Model emphasizes risk analysis.
- Agile supports continuous improvement.
- Models improve project planning.
- They reduce development risks.
- Software quality improves.
- Team collaboration becomes better.
- Appropriate model selection leads to project success.

Question 10: Construct a Concept Map

Given:

Class Diagram

→ Class

→ _____

→ Association

→ _____

Answer

Completed Concept Map

Class Diagram

|

|— Class

|— Attribute

|— Association

|— Method

Missing Concepts

- Attribute
- Method

Explanation

A **Class Diagram** is one of the most important UML structural diagrams used in Object-Oriented Design. It represents the static structure of a software system by showing classes, their attributes, methods, and relationships.

Attribute

Attributes are the properties or data members of a class. They store information related to an object. For example, a Student class may have attributes such as Name, Roll Number, Age, and Department.

Method

Methods are the functions or operations that define the behavior of a class. They describe what actions an object can perform. Examples include `displayDetails()`, `calculateSalary()`, or `withdrawAmount()`.

Association

Association represents the relationship between two or more classes. It shows how objects interact or communicate with each other. For example, a Teacher teaches Students, or a Customer places Orders.

Analysis / Importance

- Class diagrams represent the static structure of software.
- Attributes define the characteristics of objects.
- Methods define object behavior.
- Associations describe relationships among classes.
- Class diagrams simplify software design.
- They improve communication among developers.
- UML documentation becomes easier.
- Software maintenance is simplified.
- Object-oriented analysis becomes more effective.
- They are widely used in real-world software development.

Question 11: Construct a Concept Map

Given:

Software Engineering Principles

→ Modularity

→ _____

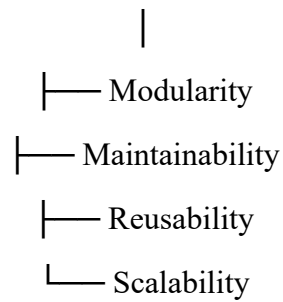
→ Reusability

→ _____

Answer

Completed Concept Map

Software Engineering Principles



Missing Concepts

- Maintainability
- Scalability

Explanation

Software Engineering Principles are fundamental guidelines that help developers build software that is reliable, efficient, maintainable, and adaptable to future changes. These principles improve software quality and reduce development and maintenance costs.

Maintainability

Maintainability is the ability of a software system to be easily modified, corrected, updated, or enhanced after it has been deployed. During the software lifecycle, requirements may change, bugs may be discovered, or new features may need to be added. A maintainable software system allows developers to perform these changes quickly and efficiently without affecting the overall functionality of the application.

Good maintainability is achieved through:

- Writing clean and readable code.
- Using proper documentation.

- Following coding standards.
- Designing modular software.
- Keeping the code well-organized and reusable.

Software with high maintainability reduces maintenance time and cost while increasing the software's overall lifespan.

Scalability

Scalability refers to the ability of software to handle increasing amounts of work, users, or data without affecting its performance. As organizations grow, software applications must support more users, larger databases, and higher transaction volumes.

A scalable system can:

- Handle increased user traffic.
- Process larger amounts of data.
- Support additional features.
- Maintain good performance during heavy workloads.
- Expand with minimal modifications.

Scalability is essential for modern web applications, cloud computing systems, banking applications, and large enterprise software.

Analysis / Importance

- Modularity divides software into smaller manageable modules.
- Maintainability simplifies software modification and bug fixing.
- Reusability reduces development time by using existing components.
- Scalability allows software to support future growth.
- These principles improve software quality and reliability.
- Development and maintenance costs are reduced.
- Team collaboration becomes more effective.
- Software becomes easier to test and upgrade.
- Customer satisfaction increases because the software remains reliable over time.
- Following Software Engineering principles results in robust, efficient, and long-lasting software systems.

Question 12: Identify the Failure Phase

Given:

Requirement Analysis ✓

System Design ✓

Coding ✓

Integration ✗

Deployment ✗

Answer

Failure Phase

Integration Phase

Explanation

The **Integration Phase** is the stage in the Software Development Life Cycle (SDLC) where individual software modules developed by different programmers or teams are combined into a single complete system. The purpose of integration is to ensure that all modules communicate correctly and function together as expected.

In the given concept map, the Integration phase has failed. This indicates that although each module was successfully designed and coded, they are unable to work together properly. The failure may occur because of incompatible interfaces, incorrect data exchange, API mismatches, configuration problems, database connection issues, or communication errors between modules.

Since the Integration phase is unsuccessful, the software cannot proceed to the Deployment phase. If integration issues are ignored, users may experience unexpected errors, system failures, or incomplete functionality after release.

Proper integration ensures that the entire software system operates smoothly as a single application rather than as separate independent modules.

Analysis / Impact

- Integration failure prevents modules from working together.
- Communication errors occur between different software components.
- Functional inconsistencies appear throughout the application.
- Deployment is delayed until all issues are resolved.
- Debugging becomes more difficult because multiple modules are involved.
- Project cost and development time increase.

- Software reliability decreases.
- User confidence is affected if defects remain unresolved.
- Additional testing is required after fixing integration issues.
- Successful integration is essential for delivering high-quality software.

Solution

To overcome Integration phase failures, developers should:

- Perform **Integration Testing** after combining modules.
- Verify that all module interfaces are compatible.
- Test APIs and database connections thoroughly.
- Resolve communication and compatibility issues before deployment.
- Use version control systems to manage module updates.
- Conduct Regression Testing to ensure previous functionalities are not affected.
- Maintain proper documentation of interfaces and dependencies.
- Follow standardized integration procedures and coding practices.

Conclusion

Software Engineering principles and the Software Development Life Cycle (SDLC) are essential for developing high-quality software systems. Principles such as **Modularity, Maintainability, Reusability, and Scalability** help developers create software that is reliable, flexible, and easy to maintain. Similarly, every SDLC phase—from **Requirement Analysis** to **Deployment**—plays a crucial role in ensuring successful software development. Identifying failures during phases like **Coding** or **Integration** helps developers correct issues early, reducing project risks and improving software quality. Following these concepts results in efficient, maintainable, and user-friendly software applications that meet customer requirements and industry standards.